SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

Course Code: ITA0609 Student Name: Shanuusree M

Course Title: Machine Learning Reg. No : 192412319

CO2 ASSESSMENT TOOL 2 -DEBUGGING TASK

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

```
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# Create linearly separable dataset
X, y = make_classification(n_samples=100,
                           n_features=2,
                           n_redundant=0,
                           n_clusters_per_class=1,
                           random_state=42)
# Train perceptron
model = Perceptron(max_iter=1000,
                   eta0=0.1,
                   random_state=42)
model.fit(X, y)
pred = model.predict(X)
print("Accuracy:", accuracy_score(y, pred))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
# Debugging:
# Change eta0 to 5 or 10 and observe poor convergence.
```

```
Accuracy: 1.0
Weights: [[-0.2523054   0.40448998]]
Bias: [0.2]
```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

```python
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = MLPClassifier(hidden_layer_sizes=(100,),
                      learning_rate_init=0.001,
                      max_iter=300,
                      random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Loss:", model.loss_)
# Debugging:
# Increase learning_rate_init to 1.0
# Observe unstable convergence.
```

```
Accuracy: 0.9833333333333333
Loss: 0.0029446643595212383
```

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

```python
import random
def fitness(x):
    return x*x
population = [random.randint(0,10) for i in range(6)]
for generation in range(10):
    population = sorted(population,
                        key=fitness,
                        reverse=True)
    print("Generation",generation,
          population)
    parent1,parent2 = population[:2]
    child = (parent1 + parent2)//2
    # Mutation
    if random.random()<0.2:
        child += random.randint(-1,1)
    population[-1]=child
print("Best Solution:",population[0])
# Debugging:
```

```
Generation 0 [10, 9, 9, 8, 8, 0]
Generation 1 [10, 9, 9, 9, 8, 8]
Generation 2 [10, 9, 9, 9, 9, 8]
Generation 3 [10, 9, 9, 9, 9, 9]
Generation 4 [10, 9, 9, 9, 9, 9]
Generation 5 [10, 9, 9, 9, 9, 9]
Generation 6 [10, 9, 9, 9, 9, 9]
Generation 7 [10, 9, 9, 9, 9, 9]
Generation 8 [10, 10, 9, 9, 9, 9]
Generation 9 [10, 10, 10, 9, 9, 9]
Best Solution: 10
```

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
(X_train,y_train),(X_test,y_test)=mnist.load_data()
X_train=X_train.reshape(-1,784)/255
X_test=X_test.reshape(-1,784)/255
y_train=to_categorical(y_train)
y_test=to_categorical(y_test)
model=Sequential()
model.add(Dense(512,activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10,activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train,y_train,
          epochs=5,
          batch_size=128)
loss,acc=model.evaluate(X_test,y_test)
print("Accuracy:",acc)
# Debugging:
```

```
y_test=to_categorical(y_test)
model=Sequential()
model.add(Dense(512,activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10,activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train,y_train,
          epochs=5,
          batch_size=128)
loss,acc=model.evaluate(X_test,y_test)
print("Accuracy:",acc)
# Debugging:
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9028 - loss: 0.3307
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.9539 - loss: 0.1579
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.9645 - loss: 0.1187
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 12s 22ms/step - accuracy: 0.9706 - loss: 0.0971
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.9743 - loss: 0.0829
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9788 - loss: 0.0686
Accuracy: 0.9787999987602234
```

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
X,y=load_iris(return_X_y=True)
model=DecisionTreeClassifier(max_depth=None)
model.fit(X,y)
pred=model.predict(X)
print("Training Accuracy:",
      accuracy_score(y,pred))
# Optimization
pruned=DecisionTreeClassifier(max_depth=3)
pruned.fit(X,y)
print("Pruned Tree Depth:",
      pruned.get_depth())
# Debugging:
# Compare deep tree and pruned tree.
```

```
Training Accuracy: 1.0
Pruned Tree Depth: 3
```

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.datasets import mnist
(X_train,y_train),(X_test,y_test)=mnist.load_data()
X_train=X_train.reshape(-1,28,28,1)/255
X_test=X_test.reshape(-1,28,28,1)/255
model=Sequential()
model.add(Conv2D(32,(3,3),
                 activation='relu',
                 input_shape=(28,28,1)))
model.add(MaxPooling2D())
model.add(Flatten())
model.add(Dense(10,  activation='softmax'))
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train,y_train,
          epochs=3)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 26s 13ms/step - accuracy: 0.9366 - loss: 0.2238
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 34s 18ms/step - accuracy: 0.9765 - loss: 0.0835
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 27s 14ms/step - accuracy: 0.9815 - loss: 0.0625
<keras.src.callbacks.history.History at 0x7f0ebf7abfe0>
```

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameter.

```
import gymnasium as gym
# Create environment
env = gym.make("FrozenLake-v1")
# Run 5 episodes
for episode in range(5):
    state, info = env.reset()
    done = False
    total_reward = 0
    while not done:
        # Random action
        action = env.action_space.sample()
        # Take action
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        state = next_state
        total_reward += reward
    print(f"Episode {episode+1}: Reward = {total_reward}")
env.close()
```

```
Episode 1: Reward = 0
Episode 2: Reward = 0
Episode 3: Reward = 0
Episode 4: Reward = 0
Episode 5: Reward = 0
```

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
X, y = make_blobs(n_samples=300,
                  centers=3,
                  cluster_std=1.2,
                  random_state=42)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KMeans(n_clusters=3,
               init='k-means++',
               random_state=42,
               n_init=10)
score = silhouette_score(X_scaled, model.labels_)
print("\nSilhouette Score:", score)
gging Tips:")
print("- Use K-Means++ initialization.")
print("- Normalize features.")
print("- Select K using Silhouette Score.")
print("- Remove outliers before clustering.")
```

```
Cluster Centers:
[[-0.21518875  1.15316649]
 [-1.06790185 -1.25264527]
 [ 1.2830906   0.09947878]]

Silhouette Score: 0.8182412709664815
```

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_scoret
X, y = make_circles(n_samples=500,
                    noise=0.5,factor=0.4,random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
linear_model = SVC(kernel='linear')
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
print("Linear Kernel Accuracy:",
      accuracy_score(y_test, linear_pred))
rbf_model = SVC(kernel='rbf',
                gamma='scale',
                C=1)
rbf_model.fit(X_train, y_train)
rbf_pred = rbf_model.predict(X_test)
print("RBF Kernel Accuracy:",
      accuracy_score(y_test, rbf_pred))
print("- Scale input features.")
print("- Choose correct kernel.")
print("- Tune C and Gamma.")
print("- Use cross-validation.")
```

```
Linear Kernel Accuracy: 0.43
RBF Kernel Accuracy: 1.0
```

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
X = np.random.randint(100, size=(100, 10))
y = np.random.randint(2, size=(100,))
model = Sequential([
    Embedding(100, 8),
    LSTM(16),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(X, y, epochs=3)
loss, acc = model.evaluate(X, y)
print("Accuracy:", acc)
print("\nDebugging:")
print("- Use LSTM instead of SimpleRNN.")
print("- Tune learning rate.")
print("- Increase training data.")
```

```
Epoch 1/3
4/4 ━━━━━━━━━━━━━━━━━━━━ 6s 35ms/step - accuracy: 0.5700 - loss: 0.6930
Epoch 2/3
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - accuracy: 0.6500 - loss: 0.6919
Epoch 3/3
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.6500 - loss: 0.6910
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.6600 - loss: 0.6903
Accuracy: 0.6600000262260437
```